SkyRoute：面向城市无人机配送的时空数据管理系统设计

SkyRoute: Spatio-temporal Data Management for Urban UAV Delivery

INT6164 课程大作业　小组报告

2026 年 11 月　版本 v2.0

角色	缩写	姓名	学号
成员 1	LJ		
成员 2	ZQO		
成员 3	LQ		
成员 4	YHT		
成员 5	ZC		

摘要

低空物流的航线查询依赖三类持续变化的数据：城市三维环境按季度更新，临时空域限制按分钟发布，地面暴露与风险按月重算。若一次查询分批读取，航线可能同时使用新空域和旧风险，结果既不可信也无法复现。SkyRoute 把这一问题定义为数据管理问题：如何设计支持高效时空查询、一致性发布、历史追溯和安全访问的数据管理系统，为航线规划提供可靠输入。

方案以一套 34 个实体的统一关系模式为核心：五类数据绑定为不可变发布，逻辑发布通过组件映射共享未变化的图层；时空 R*Tree 支持“任务窗口内的有效限制”等查询；发布事务以写锁、预期版本比较和幂等凭据保证原子切换；风险数据作为带模型版本的派生数据集封存；查询结果、航段来源与派生图缓存按发布记账，可追溯到来源与模型版本。主要创新在于把数据血缘与版本化派生数据管理落实为数据库约束和可复核实验。

在 SQLite 原型上的合成实验表明：四维时空索引在 1 万条限制、20 个历史发布时每次查询 37.4 µs，全表扫描为 12.4 ms；1000 次发布的组件共享比全量复制节省 81.7% 存储；规划服务经时空索引取数 0.04–0.17 ms，比整份 JSON 快约 200 倍；新发布后按数据库变更记录沿用或修补派生图，比全部重建快 6.9 与 16.2 倍且结果完全一致；备份恢复后的完整性、外键、发布链与历史重放核验能发现截断损坏。

关键词：时空数据管理；版本化发布；数据血缘；派生数据管理；空间索引；无人机物流

目录

1 应用场景与问题定义 **4**
1.1 背景与核心问题 . . . 4
1.2 用户与价值 . . . 4
1.3 范围与假设 . . . 4

2 需求与数据分析 **5**
2.1 功能与非功能需求 . . . 5
2.2 数据来源、规模与增长 . . . 6
2.3 生命周期与访问模式 . . . 6

3 系统与数据架构 **7**
3.1 总体架构 . . . 7
3.2 数据流与版本边界 . . . 7
3.3 部署、备份与恢复 . . . 8

4 数据模型与数据库设计 **8**
4.1 统一模式 . . . 8
4.2 约束：把规则放进数据库 . . . 9
4.3 索引、分区与冷热分层 . . . 9
4.4 操作 A：任务窗口内的有效限制 . . . 10
4.5 设计权衡：灵活模式与数据质量 . . . 11

5 事务、版本与派生数据管理 **11**
5.1 操作 B：发布与撤销 . . . 11
5.2 历史存储：组件共享与全量复制 . . . 11
5.3 新兴技术：数据血缘与版本化派生数据管理 . . . 12
5.4 设计权衡：一致性与可用性 . . . 13

6 查询负载与技术选型 **13**
6.1 规划查询读取什么 . . . 13
6.2 固定 A*，只换数据访问方式 . . . 13
6.3 物化派生图与新发布后的失效处理 . . . 14
6.4 技术比较与选型 . . . 14
6.5 设计权衡：物化与新鲜度 . . . 15

7 数据治理、安全、隐私与伦理 **15**
7.1 质量与版本化派生数据 . . . 15
7.2 血缘与操作 C：追溯一条航线的来源 . . . 15
7.3 权限、审批与审计 . . . 16
7.4 隐私、合规与伦理 . . . 16
7.5 设计权衡：共享价值与隐私保护 . . . 16

8 评估与设计权衡 **17**
8.1 实验汇总 . . . 17
8.2 恢复验收与成本 . . . 18
8.3 设计权衡汇总 . . . 18
8.4 局限与改进方向 . . . 18

9 小组分工说明 **19**

A 航线算法对照 **21**

B 复现与证据 **21**

1 应用场景与问题定义

1.1 背景与核心问题

城市无人机配送需要在高层建筑、商业区、住宅区和人员密集区域之间规划三维航线。航线是否可行取决于三类数据：建筑与地形决定空间障碍，空域管理机构发布的禁飞与限高决定某一时段能否通行，人口与设施暴露决定地面风险。三类数据来源不同、更新节奏不同、敏感程度也不同。建筑一个季度更新一次，临时禁飞可在几分钟内生效，暴露数据按月重新编译。

在这种环境下，航线算法本身不是瓶颈。真正的风险是数据层：查询若在空域更新的同时分批读取，可能把新禁飞区与旧风险值拼在一起；结果写入后，管理员也无法说清这条航线依据的是哪一版数据、哪一个风险模型。本项目的核心研究问题是：

> **城市三维空间、动态空域和风险数据持续变化时，如何设计一个支持高效时空查询、一致性发布、历史追溯和安全访问的数据管理系统，为航线规划提供可靠输入？**

航线规划在本设计中承担业务场景的角色：它是数据库最重要的读负载，用来检验数据模型、索引、事务与治理设计是否可靠。

1.2 用户与价值

表 1：目标用户与数据管理价值

用户	需要的结果	数据管理价值
物流调度员	查询可行航线并确认所用版本	不会采用已被新限制影响的旧结果
无人机运营人员	查看限制、风险依据与数据质量	识别未知数据和需要复核的区域
空域及安全管理员	发布、撤销限制并追溯影响	变更责任、事件顺序与历史状态完整保留
系统管理人员	审计、恢复与版本核验	故障后能证明数据与发布清单一致

此外，数据提供方关心来源与许可不被混用，居民关心风险表达不被误读、个人信息不外泄。这两类利益相关者的要求分别落实在来源登记（第 4 节）与隐私设计（第 7 节）。

1.3 范围与假设

研究对象是包含高层建筑、商业区、住宅区及人员密集区域的模拟城市片区。空间采用局部米制直角坐标（PostGIS SRID 0），垂直基准为片区共同参考平面；时间统一为 UTC，有效期采用半开区间 $[lo, hi)$。全部数据为合成数据。原型使用 SQLite（WAL 模式）实现并实测；部署目标为 PostgreSQL 与 PostGIS，作为设计论证。真实飞控、实时自主避障与实际飞行部署不在范围内。课程依据为《作业要求.pdf》第 1–2、6–7 页 [1]。

2 需求与数据分析

2.1 功能与非功能需求

表 2: 功能需求

编号	需求	输入	输出与验收
F1	Q1 三维航线查询	起终点、任务窗口、规划配置、发布	航线或不可行/未知、原因、度量、所用发布与数据集
F2	Q2 航段/区域状态与来源	三维航段或区域、时刻、发布	允许/禁止/未知、限制与有效期、风险值、来源与审计线索
F3	空域维护	限制体、生效窗口、发布或撤销	新发布与空域事件；旧发布只读保留
F4	数据发布	建筑、地形、空域、暴露输入	质量门禁后的不可变发布与覆盖清单
F5	查询审计与追溯	查询参数、版本、结果	可追溯的结果记录，不存完整敏感轨迹
F6	权限控制	角色、资源、敏感级别	最小权限读取、发布与审批；拒绝留有原因

Q1 与 Q2 的状态优先级为：命中硬约束返回不可行或禁止；关键数据缺失、覆盖不完整或版本不一致返回未知；两者都不成立才返回可行。未知不能当作安全值。

表 3: 非功能需求

维度	目标	说明
延迟	2,400 栋建筑规模下 Q2 p95 不超过 200 ms；Q1 取数不超过 50 ms	设计目标；第 8 节给出实测
新鲜度	空域事件分钟级可见；建筑季度、地形年度、暴露月度	发布延迟与源数据延迟分开记录
一致性	每次查询绑定一个发布；五层版本与风险模型版本一致	不跨发布拼接对象
可追溯	任意结果可回到对象、来源、发布、模型与审计事件	对象级来源与发布清单
可用性	读路径可用 99.5%，发布失败可回滚	单机原型不验证高可用，部署见第 3.3 节
合规	分级访问、最小化、保留期可配置	治理策略见第 7 节

表 4: 三类数据（三年数字为算术估算，非实测）

数据	来源与格式	更新频率	基线 / 三年假设	主要质量风险
城市三维环境	城市 GIS、BIM、LiDAR；GeoJSON、IFC、LAS	建筑季度或年度，地形年度	25 栋 / 约 2,400 栋	高度缺失、轮廓自交、坐标与高程基准不一致
动态空域	空域机构接口或人工发布；JSON	事件驱动，分钟级	3 个限制体 / 约 100 条活动限制，每日 20 条事件	生效时间歧义、撤销后复用标识、发布与查询交错
地面暴露与风险	人口、POI、土地利用；LQ 派生	月或季度，模型评审时	6 个分区 / 60 个分区	暴露与后果假设不透明、分区漏覆盖、代理值被误读为概率

2.2 数据来源、规模与增长

按每日发布一版、保留三年估算，2,400 栋建筑约 1,095 个发布，原始发布数据约 0.97 GB，加索引与历史约 2 GB；空域事件三年约 2.2 万条；查询审计按每日 1 万次约 11 GB，热库只保留 90 天。

2.3 生命周期与访问模式

数据生命周期为“采集—暂存—质检—发布—查询—归档”。采集登记来源、许可、精度与获取时间；暂存层允许灵活字段；质检检查结构、几何、属性、时间、覆盖和版本；发布生成五层版本与覆盖清单；查询只读绑定发布；归档保留只读发布与血缘。

表 5: 主要访问模式

访问模式	频率	访问路径与设计
Q1 航线查询	高	固定发布，按三维走廊与任务窗口取障碍、限制、风险；时空索引加精确复核
Q2 航段/区域查询	高	按发布、时刻与航段取限制、风险与来源
发布与撤销	低	写新发布与事件；事务、幂等、失败回滚与审计
质检与批处理	中	全量或按瓦片扫描；分区裁剪、错误隔离
审计与复现	低	按请求、发布、时间检索；只追加、按月分区、冷热分层

3 系统与数据架构

3.1 总体架构

图 1 给出采集、存储、处理、服务与消费五层。数据库中的源数据、发布清单、组件映射、派生风险与审计是权威记录；JSON 导出、空间索引和规划派生图都可由发布重建。

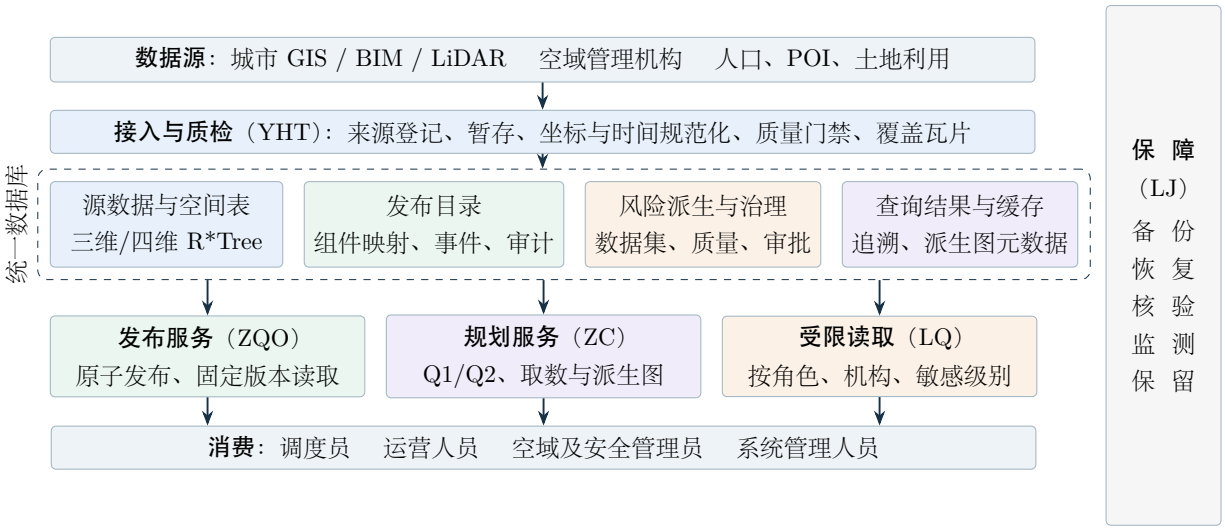


图 1: 总体架构。颜色表示主责成员；虚线框内为同一数据库中的四个表组。

各层职责如下。接入层（YHT）把多源数据规范到统一坐标、时间与质量字段，未通过门禁的数据不能进入发布。存储层把源数据、发布目录、风险派生与查询结果放在同一数据库，使一次发布能在一个事务中提交。处理层（LQ）在发布后编译任务窗口的风险数据集并封存。服务层由发布服务（ZQO）、规划服务（ZC）和受限读取（LQ）组成。保障层（LJ）负责备份、恢复核验、监测与保留。

3.2 数据流与版本边界

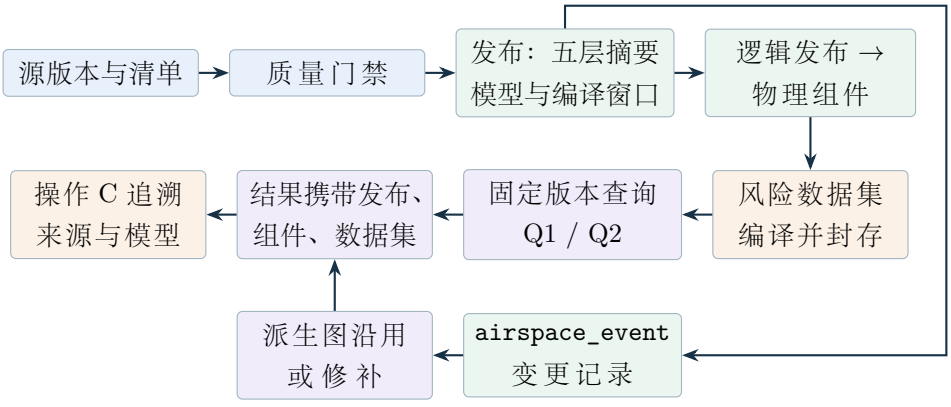


图 2: 数据流与版本边界。查询在一个读事务内固定发布，结果记录所用的发布、组件与风险数据集；新发布的变更记录驱动派生图更新。

源版本与查询版本分别保存：源版本说明上游交付了什么，层版本以规范化内容摘要标识实际写入的关系行。发布同时记录来源清单、风险模型、编译器与编译窗口，窗口不匹配即拒绝发

布。查询先固定逻辑发布，再经组件映射找到每层的物理所有者；直接按 `feature.release_id` 过滤逻辑发布会漏掉共享图层。

3.3 部署、备份与恢复

课程原型采用单机 SQLite WAL：统一模式、发布目录、审计与查询追溯表在同一数据库中。生产目标为 PostgreSQL 与 PostGIS 主库加只读副本，规划服务在内存中保留短期派生图缓存；边缘节点只缓存已发布的只读快照，发布与权限变更回到主库完成。

数据库文件、WAL、审计事件与发布哈希作为一个整体保护；内存中的派生图不备份，恢复后按发布重建。备份流程为：记录操作编号、当前指针与 WAL 大小；使用 SQLite 在线备份 API 复制数据库 [8]，或在 checkpoint 后把数据库与 WAL 作为同一备份集合；备份摘要与时间写入备份记录，副本进入加密对象存储。恢复到隔离路径后依次执行完整性检查、外键检查、发布父链与内容哈希检查，再重放一条历史查询并比对保存的结果摘要，全部通过才切换读流量；任一检查失败即进入阻断状态，不得标记为可用。

单机原型的设计目标为 RPO 不超过 15 分钟、RTO 不超过 30 分钟。RPO 取决于 WAL 与备份频率，RTO 取决于文件恢复、核验与派生图重建的规模，部署前需要按目标规模演练。空域发布期间优先保留最后一个已验收发布：宁可暂停新发布与受影响的 Q1，也不返回版本混合的结果。

4 数据模型与数据库设计

4.1 统一模式

逻辑模型以 YHT 的数据字典为准，可执行模式为一份 SQLite DDL，含 34 个实体、主外键、`CHECK` 约束、生成列、三个 R*Tree 虚表与触发器。表 6 按负责人列出表组，图 3 给出核心实体关系。

表 6: 统一模式的表组

表组	负责	实体
源数据与空间	YHT	source、coverage_tile、feature（建筑、空域限制、暴露分区三个子类型）、terrain_surface、terrain_post
发布与版本	ZQO	release、release_layer、layer_component、release_component、component_source、current_release、airspace_event、publish_request、failed_attempt、outbox
风险派生与访问	LQ	risk_model、risk_model_control、risk_dataset、risk_input_period、derived_risk、quality_check、actor、access_policy、lineage_edge、audit_event、governance_request、governance_approval
查询与结果追溯	ZC	query_run、route_result、route_segment_source、route_graph_cache

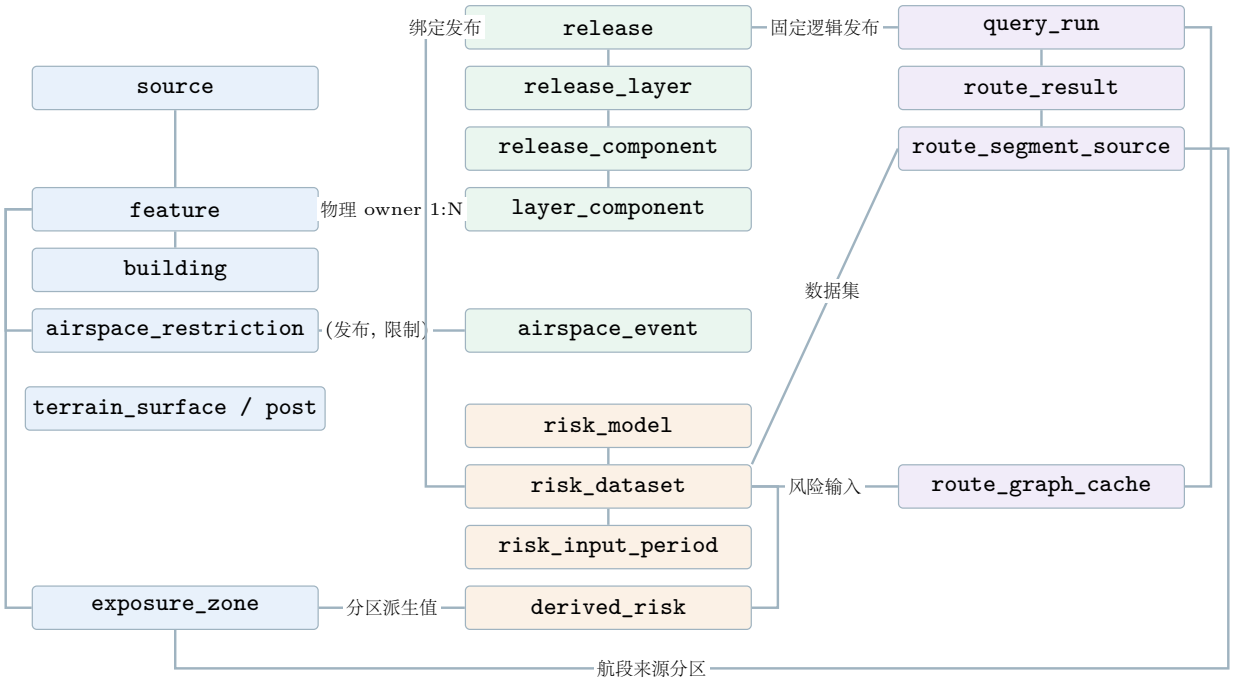


图 3: 核心实体关系（节选）。蓝：源数据（YHT）；绿：发布（ZQO）；橙：风险派生（LQ）；紫：查询与缓存（ZC）。完整 34 个实体与外键见仓库 `docs/v2/schema/er.md`。

4.2 约束：把规则放进数据库

关键规则由数据库在写入时强制，不依赖应用自觉：

- **发布不可变。**只能向 draft 发布写入内容；发布后对其空间行、子类型、地形与覆盖瓦片的插入、修改和删除一律被触发器拒绝；状态只能按 draft→published→superseded/retired 迁移；发布前五层必须齐全。
- **来源与模型不可变。**被已发布内容引用的来源行和已批准的风险模型不能原地修改，许可或参数修订必须登记新标识，旧发布仍解析到旧行。
- **取值一致。**建筑高度等于顶底差；暴露值满足 $q = E \times C$ 且 $0 \le E, C, q \le 1$；限高体必须给出上限；有效期 $lo < hi$；子类型必须与公共头一致。
- **只追加。**空域事件、审计、失败记录、风险派生、查询结果与派生图缓存行禁止修改和删除。

建模过程中修正了三处问题：覆盖瓦片和地形编号在每次发布重复出现，主键必须包含发布编号，否则第二次发布即主键冲突；空域限制的状态随发布固定，撤销写入新发布而不原地修改；限高体禁止的是上限以上的空间，索引与复核使用生成列表示的禁止高度范围，否则会漏掉越过限高的航线。

4.3 索引、分区与冷热分层

建筑使用三维 R*Tree，空域限制使用四维 R*Tree（x, y, z, t，其中 z 为禁止高度范围），暴露分区使用二维 R*Tree。R*Tree 以 32 位浮点向外取整存储，只产生候选，精确判定在基表列上复核 [2]。部署到 PostGIS 时，三维体使用 n 维 GiST 索引与 `&&&` 候选过滤，精确判定使用 `ST_3DIntersects`；有效期使用范围类型的 GiST 索引；审计使用 BRIN [9, 10, 11, 12, 13]。

空间对象表按发布编号哈希分区，查询总带发布编号，可裁剪到单一分区；审计与空域事件

按月范围分区 [14]。当前发布与最近 90 天审计留在热库，被替代的发布转入列式文件或对象存储，只保留发布层、清单哈希与血缘在热库。覆盖瓦片是数据完整性单位，用于质检与并行加载，与物理分区相互独立。

4.4　操作 A：任务窗口内的有效限制

操作 A 回答“这次任务的时间窗口内，这条三维走廊会碰到哪些有效限制”，是 Q1 取数的第一步：

```
SELECT f.feature_id, ar.kind, ar.floor_z, ar.ceiling_m
FROM rtree_airspace AS r                                   -- x, y, z, t
CROSS JOIN airspace_restriction AS ar ON ar.rid = r.rid    -- fix join order
CROSS JOIN feature AS f ON f.rid = ar.rid
WHERE r.max_x >= :x0 AND r.min_x <= :x1 AND r.max_y >= :y0 AND r.min_y <= :y1
  AND r.max_z >= :z0 AND r.min_z <= :z1
  AND r.max_t >= unixepoch(:win_lo) - 1767225600
  AND r.min_t <= unixepoch(:win_hi) - 1767225600
  AND f.release_id = :owner AND ar.state IN ('published', 'active')
  AND ar.effective_lo < :win_hi AND ar.effective_hi > :win_lo   -- exact recheck
  AND ar.forbid_hi_z >= :z0 AND ar.forbid_lo_z <= :z1;
```

执行计划为 `SCAN r VIRTUAL TABLE INDEX 2:D1B0D3B2D5B4D7B6`，随后按主键回表 [3]。实验在同一批合成数据上比较五种固定路径与 SQLite 规划器自选，六种路径对每条查询的结果完全一致（图 4）。

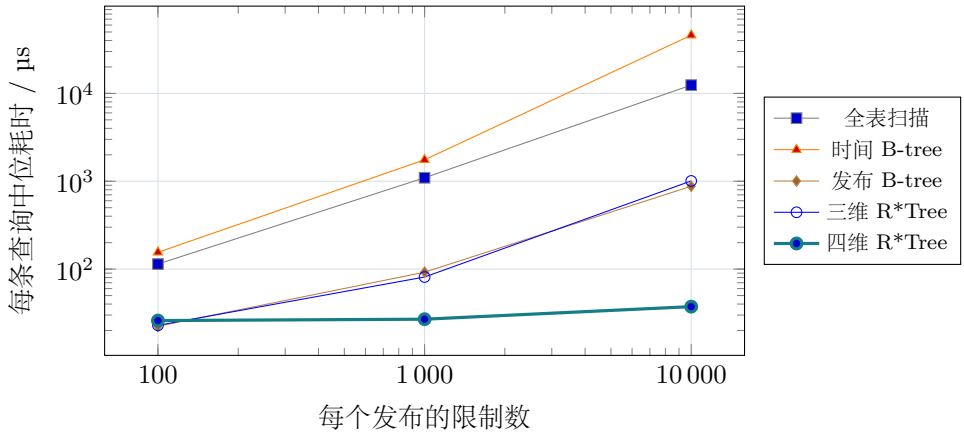


图 4: 操作 A 的访问路径（表中保存 20 个全量复制的历史发布；内存 SQLite 3.54.1，每组 100 条查询）。

结论有三点。第一，约 100 条活动限制且只存当前发布时，所有路径都在 17–30 µs，索引选择影响很小。第二，限制增多或历史留在同一张表后，四维 R*Tree 保持在 26–38 µs；1 万条限制、20 个发布时为 37.4 µs，发布 B-tree 为 878 µs，全表扫描为 12.4 ms。第三，单列时间 B-tree 只能按开始时间单侧裁剪区间，候选约为全表一半，加上回表比全扫更慢；PostgreSQL 的范围类型 GiST 可以解决这一点 [11, 12]。规划器自选的中位数接近最优，但会随参数在不同路径间切换，1 万条限制时 p95 达 781 µs，因此查询用 `CROSS JOIN` 固定由 R*Tree 驱动 [4]。

4.5 设计权衡：灵活模式与数据质量

暂存层接收多源 GIS 字段时允许灵活模式，扩展属性存入 JSON 列；发布层只保留经过类型化、约束与质检的核心列。规则是：参与硬约束、连接、索引、权限判断或安全结论的值必须是类型化列，JSON 列只承载清单、参数、证据与扩展属性。这样新数据源不必每次改表，同时“能存进去”不会被当成“能安全查询”。纯模式读取会让质量规则散落在查询中，纯严格模式会让每次接入都需要迁移。

5 事务、版本与派生数据管理

5.1 操作 B：发布与撤销

发布在 `BEGIN IMMEDIATE` 写事务中完成：检查幂等请求、预期当前版本、发布者角色与五层输入；写入 draft 发布、组件与五层映射；为每条改动的限制追加空域事件；把发布置为 published，以比较并交换的方式切换当前指针，把旧版本置为 superseded；写成功审计、待发送事件与幂等凭据后提交 [6, 5]。任一步失败即整体回滚，并在独立事务中追加失败记录。撤销不删除旧限制，而是在新发布中写入状态为 revoked 的行与一条撤销事件。

写锁使数据库写入串行化，预期版本比较拒绝在等待写锁期间已经过时的请求；幂等凭据先于当前版本检查，提交成功但响应丢失时，同一请求重试会返回已提交的发布。WAL 读事务在发布期间继续读取旧快照，新事务读到完整新版。

表 7: 并发、可见性与回滚（文件 SQLite，WAL，FULL 同步）

实测场景	结果
两个连接竞争同一旧版本	一次成功，一次版本冲突，保留失败审计
写入未提交时与提交后的旧读事务	均看到旧版；结束读事务后看到新版
五处注入失败（draft、组件、事件、指针切换、提交前）	业务表与当前指针全部回滚，只新增失败记录
提交后丢失响应并重试	返回同一发布，不重复产生事件
撤销与历史重开	新版撤销，旧版内容与来源保持一致

5.2 历史存储：组件共享与全量复制

发布不可变带来历史版本的存储开销。逻辑发布与物理组件分离后，未变化的图层由新发布直接引用旧组件，只有变化的图层写入新行。在每版只修改一个限制高度的负载下（25 栋建筑、121 个地形点、3 个限制、6 个暴露分区），1000 次发布的数据库为 6.20 MiB，全量复制为 33.91 MiB，节省 81.7%（图 5）；抽样历史导出的内容摘要相同，历史重建中位耗时约 1.04 ms 与 1.07 ms，两者相当。

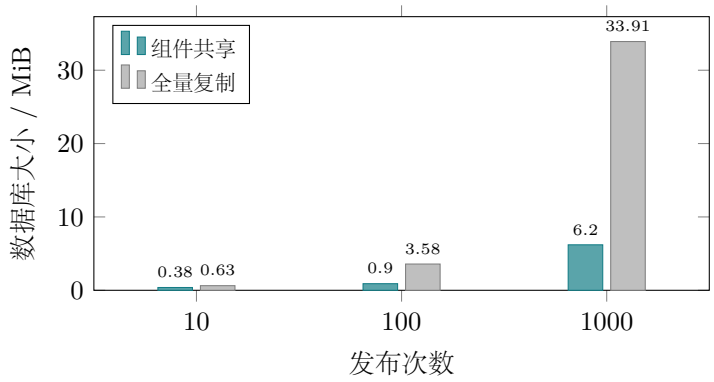


图 5: 历史存储开销（checkpoint 后测量，含索引与审计）。

5.3 新兴技术：数据血缘与版本化派生数据管理

本项目深入讨论课程列出的“数据目录、元数据管理、数据血缘与数据可观测性”方向 [1]，聚焦一个具体问题：当输入变化时，哪些派生数据需要重建，重建代价是多少。SkyRoute 中有三类派生数据：风险数据集由暴露组件、风险模型与编译窗口生成；规划派生图由发布、风险数据集、任务窗口与规划配置生成；JSON 导出由发布生成。设计采用四个机制：

1. **来源依赖可查询。**组件反向索引列出依赖某一组件的发布；`lineage_edge` 记录来源、模型、组件到风险数据集的依赖，触发器校验端点存在与依赖真实；递归查询列出受影响的数据集。
2. **派生数据带版本键。**风险数据集绑定发布、组件、模型与窗口，封存后不可变；派生图按发布、搜索域、窗口与配置记账，查询引用的图必须与查询的发布一致。
3. **变更驱动失效定位。**新发布到来时，从发布事务写入的空域事件与两份风险数据集的差异得到变更范围，只有与变更相交的派生对象需要处理（第 6 节实测）。
4. **旧派生数据永久有效。**发布不可变，按发布记账的派生数据对其自身发布始终正确，历史查询直接复用；“失效”只表示新发布不能沿用。

表 8: 版本化方案比较

方案	历史与并发机制	适用条件
SQLite 不可变组件共享（本项目）	一个事务提交五层、当前指针与审计；WAL 读快照；历史按发布编号重建	单库、可复现的设计验证；写入串行，共享粒度为整层
Apache Iceberg 快照与时间旅行	每表快照引用不可变数据文件，原子替换元数据并校验并发提交，可按版本或时刻查询 [18, 19]	批量三维资产、对象存储与多计算引擎成为主要负载时；五类数据跨表一致发布需另行设计
PostgreSQL 时态建模	范围类型与时间约束表达有效时间，历史表与触发器记录系统时间 [15, 16]	多用户细粒度更新、集中授权与 PostGIS 查询；MVCC 本身不构成长期历史库

5.4 设计权衡：一致性与可用性

发布期间旧读事务继续读取旧快照，读取不被阻塞；代价是长读事务看到的“当前”可能已被替代。设计要求调度在采用航线前用新事务复核当前指针，查询结果中的新鲜度由数据库按当前指针判定。由此，发布期间保持读取可用，同时任何被采用的结果都能说明它基于哪一版数据。

6 查询负载与技术选型

6.1 规划查询读取什么

一次 Q1 在一个读事务内完成：读取发布与当前指针，确定逻辑发布及其新鲜度；经组件映射找到各层的物理所有者；绑定一个已封存、属于该发布暴露组件、窗口覆盖任务窗口的风险数据集；用三维、四维与二维 R*Tree 读取搜索域附近的建筑、限制与暴露分区，并连接派生风险值；最后把结果写入 `query_run`、`route_result` 与 `route_segment_source`。这些表的触发器要求查询绑定已发布、映射完整、有效期覆盖任务窗口的发布，提交者为在岗业务角色，新鲜度由数据库判定，航段来源必须属于查询发布与覆盖任务窗口的数据集，所有结果只追加。

搜索域取起终点包围盒外扩 40 m、高度 20–120 m、20 m 网格，共 825–891 个节点。3 km × 3 km 合成城市在 200、2,400、10,000 栋建筑时，每个搜索域内的对象中位数分别为 4、18.5、71.5 个。

6.2 固定 A*，只换数据访问方式

实验让航线算法保持不变，同一批任务分别经四条路径取数：读取整份发布的 JSON 后在应用内过滤；SQL 读取发布全部对象后在应用内过滤；R*Tree 只返回搜索域附近的对象；复用已物化的派生图。

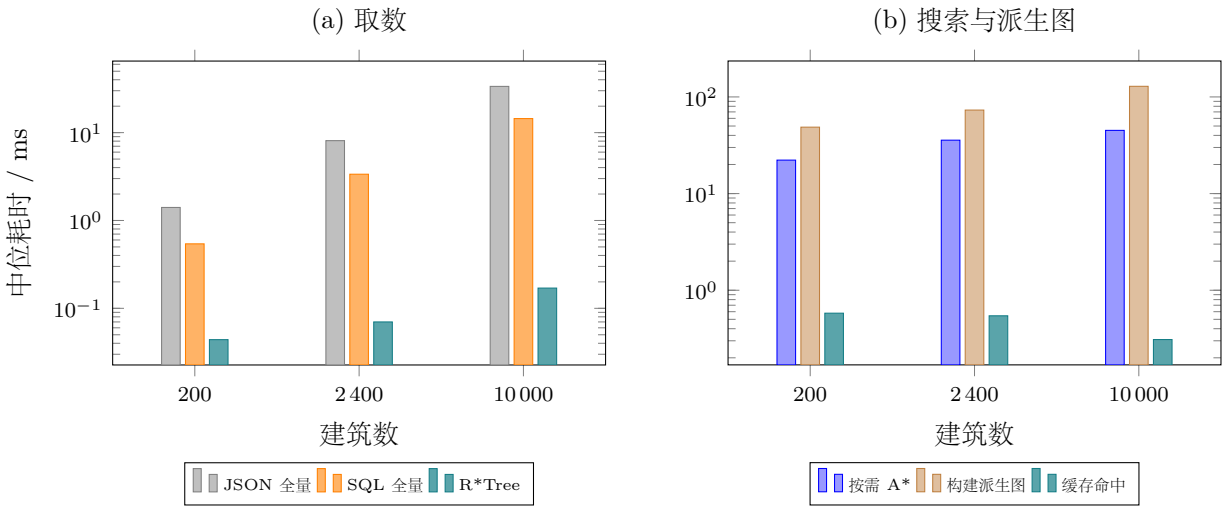


图 6: 规划查询的耗时分解（文件 SQLite 3.54.1，WAL；每规模 12 个任务）。

收益来源可以分开计量（图 6，表 9）：

- **取数成本由数据库设计决定。**整份发布的读取随城市规模线性增长，1 万栋时 JSON 路径 33.6 ms，已达单次搜索的四分之三；R*Tree 路径只与搜索域内对象数有关，中位数不超过

表 9: 规划查询各阶段中位耗时（ms）

建筑数	JSON 全量	SQL 全量	R*Tree	按需 A*	构建派生图	缓存命中
200	1.41	0.54	**0.04**	22.2	48.7	0.58
2,400	8.11	3.36	**0.07**	35.7	73.1	0.54
10,000	33.6	14.5	**0.17**	45.1	128.8	0.31

0.17 ms，比 JSON 快约 200 倍，比 SQL 全量快约 85 倍。

- **结果不受取数方式影响。**36 个任务中三条非缓存路径交给规划器的输入摘要逐字节相同，另用 4 个任务对比全城对象与过滤后对象，规划结果一致；速度差异全部来自取数路径。
- **搜索成本由算法与对象密度决定。**按需 A* 在扩展节点时逐条检查航段与对象相交，耗时随域内对象数从 22.2 ms 增至 45.1 ms；同一派生图上 A* 扩展 125–214 个节点，Dijkstra 扩展 465–747 个，最优代价 36/36 一致（附录 A）。

6.3 物化派生图与新发布后的失效处理

派生图把搜索域内全部允许航段及其代价预先算好，元数据写入 `route_graph_cache` 并绑定风险数据集。构建一张图需 49–129 ms，是一次按需搜索的 2–3 倍；命中后整条查询只需 0.31–0.58 ms，同一键的第 3 次查询起收回构建成本。

新发布到来时，搜索域与任务窗口都不与变更相交的图直接沿用，只插入一行缓存记录；相交的图只重算触及变更的节点与航段；受影响节点超过一半，或场景、地形、暴露组件发生变化时整图重建。两次真实发布的结果见表 10：每张图都与全部重建的结果比对摘要，48 张图 0 处不一致；变更检测低于 0.2 ms。新增禁飞区使一条航线由可行变为不可行，说明失效处理直接决定结果正确性。旧发布上的 24 个历史查询全部直接命中原缓存。

表 10: 新发布后派生图的处理（2,400 栋，24 张缓存图）

发布变化	沿用 / 修补 / 重建	全部重建	按变更处理	加速	与重建不一致
R0→R1 新增禁飞区	20 / 3 / 1	2,394 ms	346 ms	6.9×	0
R1→R2 撤销禁飞区	22 / 2 / 0	2,390 ms	147 ms	16.2×	0

6.4 技术比较与选型

原型采用 SQLite + R*Tree；部署目标为 PostgreSQL + PostGIS，满足多用户写入、行级权限与精确三维谓词；派生图保留在规划服务内存，依赖关系与有效性记录在数据库。理由是：航段可行性同时取决于发布、任务窗口与规划配置，持久图存储或 pgRouting 需要为每个组合物化约 9,000–17,000 条边，构建成本与派生图相同，还增加一个需要与发布语义同步的存储。取数阶段经 R*Tree 后已降到 0.2 ms 以内，剩余成本在航段检查，移入另一存储并不能减少它。大量用户反复查询少数固定走廊、限制变化较少时，持久化图的复用价值上升，可作为后续评估对象。

表 11: 技术比较

方案	时空过滤	路径查询	一致性与并发	本项目证据
SQLite + R*Tree	三维/四维 R*Tree，候选后精确复核	应用内 A*	单写多读（WAL）；发布在一个事务内	操作 A 与第 6 章实验
PostgreSQL + PostGIS	n 维 GiST 与范围 GiST	应用内 A*	MVCC 多写；行级安全承载权限	设计映射
图数据库（Neo4j 类）	依赖外部空间预处理	内置路径算法与图投影 [20]	与关系库分属两个存储，需同步发布语义	未部署
PostGIS + pgRouting	PostGIS 过滤	`pgr_aStar` 等库内搜索 [21]	与 PostGIS 同库；航段须作为边表供 SQL 读取	未部署

6.5 设计权衡：物化与新鲜度

物化派生图把单次查询从 22–45 ms 降到 0.3–0.6 ms，代价是每张图 49–129 ms 的构建、9,000–17,000 条边的内存占用以及新发布后的失效处理。缓存以发布为键，查询绑定的发布与图必须一致，因此任何查询都不会读到其他版本的图；新发布后按变更记录沿用或修补，24 张图的可用时间从全部重建的 2.4 s 降到 0.15–0.35 s。同一键查询少于 3 次、或限制频繁覆盖大面积时，按需搜索更合适。

7 数据治理、安全、隐私与伦理

7.1 质量与版本化派生数据

风险层是带模型版本的派生数据集。`risk_dataset` 绑定逻辑发布、物理暴露组件、风险模型、编译窗口与编译器；`risk_input_period` 保留时段输入，`derived_risk` 保存全窗口最大值，`quality_check` 保存检查证据。时段先裁剪到任务窗口，数据库拒绝重叠；封存时触发器独立核对每个分区覆盖的时长与最大代理值，即使应用写入“质检通过”，时间缺口也无法封存。已批准或被引用的模型禁止原地修改，修订使用新模型标识；模型启停记录在独立控制表并审计，历史解释不随停用改变。演示代理 $q = E \times C$、硬阈值 0.9 与 5 m 影响包络均为合成参数，代理值不是事故概率。人口密度是输入之一，不直接等同于安全性。

7.2 血缘与操作 C：追溯一条航线的来源

操作 C 把一条航线的每个航段连接到逻辑发布、暴露组件、物理所有者下的分区、已封存数据集、派生风险值、来源与模型版本（图 7）。正式结果表在写入时已校验组件、分区、数据集与窗口的一致性，追溯 SQL 再次连接以便独立复核；36 条实验航线的每个航段都能追溯到来源。来源或模型变化时，递归查询 `lineage_edge` 列出需要重新编译或复核的数据集，这表示候选范围，不直接判定航线禁飞。

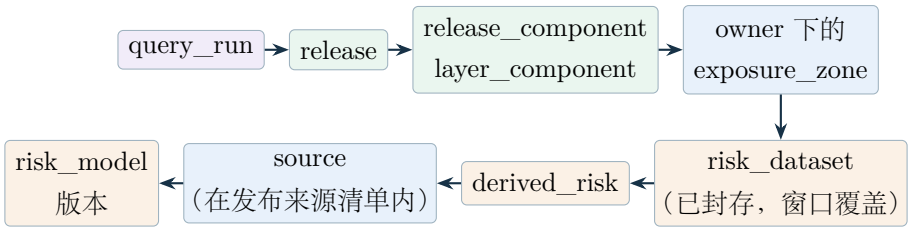


图 7: 操作 C 的追溯链。共享组件的物理所有者可以是旧发布，追溯经组件映射而非直接按逻辑发布筛选。

7.3 权限、审批与审计

表 12: 权限矩阵

用户	读取	写入	发布	审批
调度员	本机构暴露摘要视图	提交规划查询	禁止	禁止
运营人员	摘要视图及任务解释	禁止修改源表	禁止	禁止
空域及安全管理员	明细视图、模型与来源	受控草稿与质检入口	经独立审批后发布	仅审批他人请求
系统管理人员	运行元数据与受控审计	身份与策略维护	默认不授予	禁止业务自审

读取服务在连接上注册身份函数，受限视图按机构、在岗状态、策略与敏感级别过滤；固定查询端点配合 SQLite 授权回调拒绝原表、内部视图与写操作 [7]。发布请求绑定作者、候选发布与内容摘要，触发器拒绝作者自审与非在岗审批人；发布网关复算摘要，防止审批后替换内容。审计记录身份、动作、对象、发布、结果与原因，读取事件另存角色快照，不保存完整敏感轨迹；职责分离、最小权限与审计保护参考 NIST SP 800-53 的相应控制 [22]。SQLite 没有 GRANT 与行级安全，文件所有者是可信边界；部署到 PostgreSQL 时使用非表所有者的应用角色、GRANT 与行级安全策略 [17]。传输加密与备份加密属于部署设计（第 3.3 节），原型未实现文件级加密。

7.4 隐私、合规与伦理

规划内部使用精确分区；面向调度的展示只保留 200 m 格网、分区数与四舍五入的代理值，少于两个合格分区的格子不显示。这降低细节泄露，也会隐藏局部差异，不能替代规划输入，也不构成匿名保证。粒度是演示参数，真实接入需按许可、目的、地域与保留要求评审。

伦理上，风险规避可能把飞行负担转移到特定住宅区，评估应按区域统计经过次数与代理累计量。AI 可辅助发现异常的暴露值与敏感字段，但缺失、偏差与新格式会造成误报漏报；AI 建议只产生待复核标记，人工确认后才形成新的数据或策略版本。已实测的是确定性数据库约束，AI 检测准确率尚未评估。

7.5 设计权衡：共享价值与隐私保护

暴露数据的空间粒度越细，调度与公众沟通的价值越高，泄露风险也越高。本设计让规划在内部使用精确分区，对外只暴露聚合摘要，并以“每格至少两个合格分区”控制细节，代价是可显

示区域减少、局部差异被平滑。

8 评估与设计权衡

8.1 实验汇总

表 13: 数据库实验汇总（全部为合成数据，结果文件记录环境与源码摘要）

实验	负责	条件	主要结果
操作 A 访问路径	YHT	内存 SQLite；每发布 100–10,000 条限制、1 或 20 个发布；6 种路径	四维 R*Tree 37.4 µs，发布 B-tree 878 µs，全扫 12.4 ms；6 种路径结果一致
并发发布与回滚	ZQO	文件 SQLite（WAL）；竞争、读写交错、五处注入失败、丢响应重试	一次成功一次冲突；未提交写入不可见；失败全部回滚；重试不重复发布
历史存储	ZQO	10/100/1,000 次发布，每版改一个限制	组件共享 6.20 MiB，全量 33.91 MiB，节省 81.7%
治理约束	LQ	6 个分区、2 个发布、2 个派生数据集；18 项测试	模型不可原地修改；时段缺口无法封存；自审被拒；受限视图按角色过滤
规划取数路径	ZC	200/2,400/10,000 栋，每规模 12 个任务	R*Tree 0.04–0.17 ms，JSON 1.4–33.6 ms；输入、结果、追溯 36/36 一致
派生图失效	ZC	2,400 栋，24 张图，两次真实发布	比全部重建快 6.9× 与 16.2×；48 张图 0 处不一致
恢复核验	LJ	在线备份；2 个发布；截断故障注入	正常恢复通过完整性、外键、发布链与历史重放；截断备份被检测
整合复核	ZC	20 组命令重跑全部测试、实验与审阅脚本	20/20 符合预期，231 项测试通过

表 14: 监测指标（2,400 栋规模实测中位数 / p95）

指标	定义	实测	建议告警
取数耗时	读事务开始到规划输入就绪（R*Tree）	0.07 / 0.09 ms	p95 > 1 ms
搜索耗时	按需 A*	35.7 / 104.9 ms	p95 > 500 ms
缓存命中耗时	查缓存到返回航线	0.54 / 1.42 ms	p95 > 5 ms
发布后缓存就绪	新发布到全部缓存沿用或修补完成	147–346 ms（24 张图）	> 5 s
追溯完整率	可行航线中能被追溯的航段比例	100%	< 100%
恢复核验	完整性、外键、发布链、历史重放	全部通过	任一失败即阻断

8.2 恢复验收与成本

恢复验收由五项检查组成：SQLite 完整性检查返回 ok，否则隔离备份并切换上一份；外键检查零行，否则阻止服务；发布父版本存在且哈希完整，否则不允许恢复为当前；固定发布的历史查询结果与摘要一致，否则标记未知并重新取证；派生图只按发布重建，不复用不匹配的图。实验对两个发布做在线备份与恢复，五项检查全部通过；截断后的备份在打开时被检测为损坏。

成本按 YHT 的规模假设估算：约 2 GB 热数据加一份同等大小的备份放在对象存储，按 2026 年 11 月查阅的 Amazon S3 Standard 公开单价 0.023 美元/GB · 月，约为每月 0.09 美元；保留 12 份月度备份约 24 GB，约为每月 0.55 美元 [23]。估算不含请求、跨区复制与出站费用，部署时需按区域重新核价。成本的主要驱动项是审计与查询结果的保留期，而非空间数据本身。

8.3 设计权衡汇总

表 15: 五项关键设计权衡

权衡	选择	获益	代价
灵活模式 vs 数据质量（YHT）	暂存灵活、发布严格	多源接入不必频繁改表，查询结构稳定	模式与质量门禁的维护成本
一致性 vs 可用性（ZQO）	不可变发布 + 读快照 + 采用前复核	发布期间读取不阻塞，结果可追溯	长读事务可能使用旧版；历史存储增加
共享价值 vs 隐私（LQ）	内部精确、对外聚合	降低敏感明细扩散	局部差异被平滑，复核需受控流程
物化 vs 新鲜度（ZC）	派生图按发布记账并按变更更新	重复查询亚毫秒级	构建与内存成本，需失效处理
恢复目标 vs 成本（LJ）	单机 WAL 备份 + 五项恢复核验	低运维、可复现	更短 RPO/RTO 需要热副本与演练，费用上升

8.4 局限与改进方向

- **部署环境未实测。**性能数据来自单机 SQLite 与合成数据；PostgreSQL、PostGIS、图数据库与 pgRouting 只作设计论证，多用户写入、复制与跨区故障切换需在目标规模上演练。
- **几何精度。**原型以轴对齐包围盒做精确判定；非矩形楼体与斜面空域需要 PostGIS 三维谓词复核。
- **风险模型。**代理值缺少机型失效率与伤害校准，不能解释为事故概率；AI 辅助治理的准确率尚未评估。
- **未关闭问题。**整合时仍有三项：LQ 的临时追溯 SQL 要求查询窗口等于数据集窗口，与正式表约束不一致（LQ-V2-02）；发布服务未拒绝空的上游版本值（ZQO-V2-01）；Windows 默认编码下的复现修复待 LJ 在 Windows 复跑确认（ZC-V2-02）。三项均不影响本报告的实验结论，跟踪记录见仓库问题清单。

- **改进方向。**在 PostgreSQL 上复现操作 A 与操作 B；为高频走廊评估持久化图；把恢复演练、保留与归档清理纳入受控流程；用真实许可数据评审隐私粒度。

9 小组分工说明

本组按课程报告章节分工，五名成员串行完成各自章节并审阅上一位成员的成果，整合与发布由 ZC 负责。各成员的实际成果如下；贡献比例由全体成员依据实际完成情况确认，并在《团队项目贡献度确认表》中逐行签字。

表 16: 成员主要贡献

成员	负责章节	主要成果
YHT	2、4	需求与数据分析；16 实体数据字典与统一逻辑模型；索引、分区与冷热分层设计；复核统一模式与操作 A 实验；修复发布后写入旁路与来源可变问题
ZQO	1、3.1–3.2、5	问题定义与总体架构；组件共享的发布目录；发布事务、并发、回滚与历史存储实验；版本化方案比较；审阅发现源数据写入旁路
LQ	7	风险派生数据集、模型不可变与质量门禁；受限视图、授权回调与审批职责分离；隐私展示粒度与伦理分析；审阅发现发布来源校验缺口
ZC	6、8.1、附录 A、整合	统一模式可执行版与操作 A 实验（代 YHT 补齐，经 YHT 复核）；查询结果与派生图缓存表组、正式追溯 SQL；规划取数与缓存失效实验；技术选型；审阅 LQ；分支与 CI；报告与汇报整合
LJ	3.3、8.2–8.3	部署与备份恢复流程；恢复核验与截断故障实验；成本估算；五项权衡汇总；审阅发现 Windows 复现问题

AI 使用说明。本组成员使用 AI 编程与写作助手（Anthropic Claude、OpenAI Codex）辅助资料检索、方案比较、代码实现、测试与文稿起草。全部实验数据由仓库脚本实际运行产生，结果文件记录运行环境与源码摘要；每名成员审阅并对本人负责章节的结论负责。课程对 AI 使用声明的具体格式以正式通知为准。

参考文献

[1] 香港教育大学, “INT6164 课程大作业：面向真实场景的新一代数据管理系统设计,”《作业要求.pdf》, pp. 1–7, 课程提供材料.

[2] SQLite, “The SQLite R*Tree Module.”[Online]. Available: `https://www.sqlite.org/rtree.html`

[3] SQLite, “EXPLAIN QUERY PLAN.”[Online]. Available: `https://www.sqlite.org/eqp.html`

[4] SQLite, “The SQLite Query Optimizer Overview.”[Online]. Available: `https://www.sqlite.org/optoverview.html`

[5] SQLite, “Isolation in SQLite.”[Online]. Available: `https://www.sqlite.org/isolation.html`

[6] SQLite, “Transaction.”[Online]. Available: https://www.sqlite.org/lang_transaction.html

[7] SQLite, “Compile-Time Authorization Callbacks.”[Online]. Available: https://www.sqlite.org/c3ref/set_authorizer.html

[8] SQLite, “Online Backup API.”[Online]. Available: https://www.sqlite.org/backup.html

[9] PostGIS Project, “Spatial Indexes,”*PostGIS 3.5 Manual*. [Online]. Available: https://postgis.net/docs/manual-3.5/using_postgis_dbmanagement.html

[10] PostGIS Project, “ST_3DIntersects,”*PostGIS 3.5 Manual*. [Online]. Available: https://postgis.net/docs/ST_3DIntersects.html

[11] PostgreSQL Global Development Group, “GiST Indexes,”*PostgreSQL Documentation*. [Online]. Available: https://www.postgresql.org/docs/current/gist.html

[12] PostgreSQL Global Development Group, “Range Types,”*PostgreSQL Documentation*. [Online]. Available: https://www.postgresql.org/docs/current/rangetypes.html

[13] PostgreSQL Global Development Group, “BRIN Indexes,”*PostgreSQL Documentation*. [Online]. Available: https://www.postgresql.org/docs/current/brin.html

[14] PostgreSQL Global Development Group, “Table Partitioning,”*PostgreSQL Documentation*. [Online]. Available: https://www.postgresql.org/docs/current/ddl-partitioning.html

[15] PostgreSQL Global Development Group, “CREATE TABLE,”*PostgreSQL 18 Documentation*. [Online]. Available: https://www.postgresql.org/docs/18/sql-createtable.html

[16] PostgreSQL Global Development Group, “Trigger Functions,”*PostgreSQL 18 Documentation*. [Online]. Available: https://www.postgresql.org/docs/18/plpgsql-trigger.html

[17] PostgreSQL Global Development Group, “Row Security Policies,”*PostgreSQL 18 Documentation*. [Online]. Available: https://www.postgresql.org/docs/18/ddl-rowsecurity.html

[18] Apache Software Foundation, “Reliability,”*Apache Iceberg Documentation*. [Online]. Available: https://iceberg.apache.org/docs/latest/reliability/

[19] Apache Software Foundation, “Queries: Time Travel,”*Apache Iceberg Documentation*. [Online]. Available: https://iceberg.apache.org/docs/latest/spark-queries/

[20] Neo4j, “Native Projection,”*Graph Data Science Manual*. [Online]. Available: https://neo4j.com/docs/graph-data-science/current/management-ops/graph-creation/graph-project/

[21] pgRouting Contributors, “pgr_aStar,”*pgRouting Manual*. [Online]. Available: https://docs.pgrouting.org/latest/en/pgr_aStar.html

[22] NIST, *Security and Privacy Controls for Information Systems and Organizations*, SP 800-53 Rev. 5, 2020. [Online]. Available: https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final

[23] Amazon Web Services, “Amazon S3 Pricing.”[Online]. Available: https://aws.amazon.com/s3/pricing/

[24] E. W. Dijkstra, “A note on two problems in connexion with graphs,”*Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[25] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,”*IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, 1968.

[26] S. Koenig and M. Likhachev, “D* Lite,”in *Proc. AAAI-02*, 2002, pp. 476–483.

网页资料访问日期均为 2026 年 11 月 6–7 日。

A 航线算法对照

规划器在有限三维网格上以 26 邻接搜索。建筑、有效限制与风险硬阈值先过滤航段，可行航段代价为“长度/100 m + λ × 风险代理”。启发式取到终点的欧氏距离除以 100 m；风险代理非负，启发式一致，A* 与 Dijkstra 得到相同的最优代价 [24, 25]。在同一派生图上运行两者，36 个任务的最优代价全部一致；A* 扩展节点中位数 125–214 个，Dijkstra 为 465–747 个，图上搜索耗时分别为 0.29–0.57 ms 与 0.77–1.73 ms。D* Lite 等增量搜索适用于同一图上的局部边变化 [26]；本设计在数据层用发布记账与变更驱动的修补处理变化，增量搜索留作后续候选。

B 复现与证据

代码与文档仓库：`https://github.com/zoran-sid/Routeweave`。在仓库根目录执行以下命令可重跑全部测试、实验与审阅脚本，输出写入临时目录，不覆盖已交付的结果文件：

```
python3 -B docs/integration/v2_verify.py --output /tmp/verify.json
python3 -B -m unittest discover -s docs/v2/schema -v
```

表 17: 证据文件

内容	仓库路径
统一模式、ER 与设计决定	`docs/v2/schema/`
操作 A 访问路径	`docs/modules/yht/v2/experiments/results/op_a_index_comparison.json`
发布事务与历史存储	`docs/modules/zqo/v2/experiments/results/transactions-storage.json`
治理约束与操作 C	`docs/modules/lq/v2/experiments/results/governance.json`
规划取数与派生图失效	`docs/modules/zc/v2/experiments/results/`
恢复核验	`docs/modules/lj/v2/experiments/results/recovery.json`
整合复核	`docs/final/evidence/verification-v2.json`
问题清单	`docs/integration/issues.md`